

Implementando a Edição

Lógica para editar o registro do Livro (Parte Teórica)

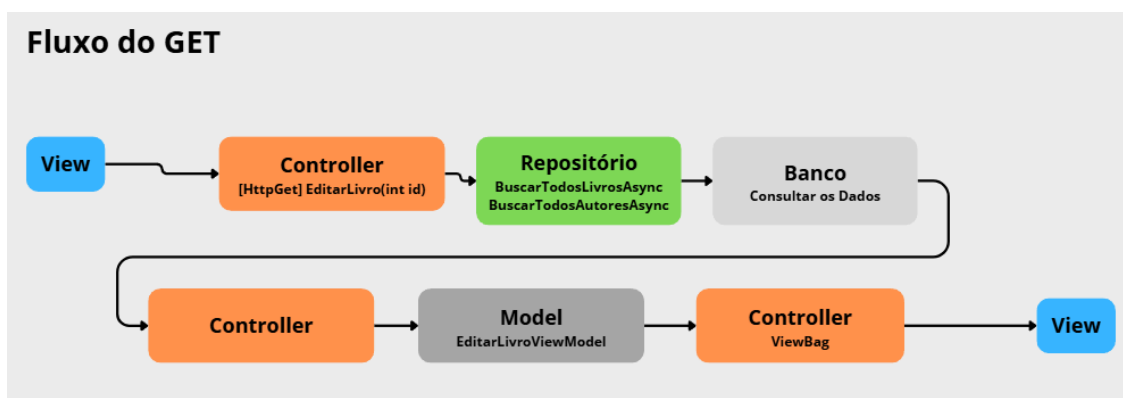
Fase 1: O fluxo do GET

Abertura da tela de edição resgatando os dados do banco

Nesta fase queremos buscar o passado do livro e exibi-lo na tela para o usuário ver o registro atual e decidir o que irá alterar.

Fluxo:

1. **O Gatilho:** O usuário está na tela de listagem e clica no botão "Editar" de um livro específico.. O navegador faz uma requisição HTTP GET para a URL `/Biblioteca/EditarLivro/[Id do livro selecionado]`.
 - a. Exemplo: Selecionou o livro com o ID 3, será feita a requisição `/Biblioteca/EditarLivro/3`
2. **A Controller recebe o pedido:** O método `[HttpGet] EditarLivro(int id)` é acionado e recebe o número correspondente ao ID selecionado.
3. **O Repositorie busca no Banco:** A Controller chama o `LivroRepository`. O repositório conversa com o Entity Framework (DbContext), que irá executar um comando SQL `SELECT` no banco de dados usando um `JOIN (.Include(x => x.Autor))` para trazer o livro e as informações do autor dele juntas.
4. **A Transformação (Model para ViewModel):** O banco devolve a entidade de domínio Livro. A Controller, por segurança, não envia essa entidade direto para a tela. Ela cria uma `EditarLivroViewModel` (uma "gaiola" de dados vazia) e joga as informações do livro lá dentro.
5. **O Dropdown é alimentado:** A Controller busca todos os autores cadastrados e cria uma `ViewBag.Autores`. Ela carimba essa lista avisando: *"O autor atual deste livro é o ID X, deixe ele selecionado"*.
6. **A View é renderizada:** A Controller entrega a ViewModel e a ViewBag para a View `EditarLivro.cshtml`. O servidor processa o HTML, preenche os inputs e gera um campo oculto (ou não) `<input type="hidden" id="Id" value="3" />`. O usuário finalmente vê a tela preenchida.



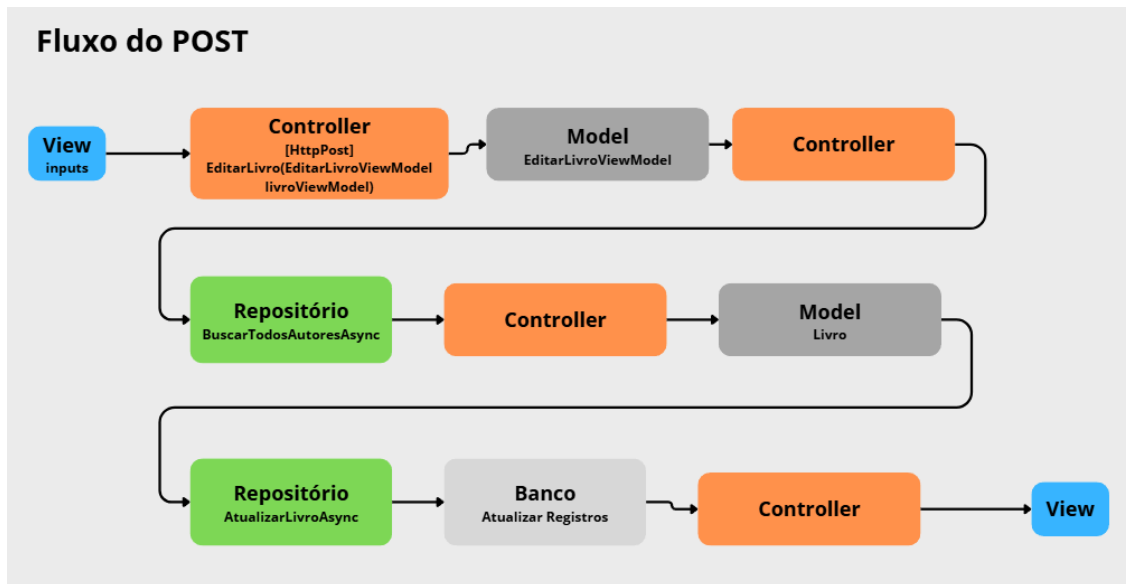
Fase 2: O Fluxo do Post

Salvando as alterações no Banco de Dados

Nesta fase queremos que quando o usuário altere os campos e clique em "Salvar Alterações", façamos o registro das atualizações no Banco.

Fluxo:

1. **O Envio do Formulário:** O navegador empacota tudo o que estava nos inputs (Título, Gênero, Páginas, Data, o ID do Autor escolhido no select e o ID do livro) e envia tudo via HTTP POST para a Controller.
2. **O Model Binding (A Mágica do ASP.NET):** O framework do ASP.NET Core intercepta esse pacote de dados brutos e os converte automaticamente em um objeto do tipo `EditarLivroViewModel`.
3. **A Reconstrução da Entidade:** O método `[HttpPost] EditarLivro` recebe essa `ViewModel` modificada. A Controller usa o `AutorId` enviado pela tela para buscar o objeto `Autor` correspondente no banco. Com isso em mãos, ela monta um objeto `Livro` novinho, associando o Autor encontrado a ele.
4. **O Repositório entra em ação:** A Controller entrega esse objeto `Livro` modificado para o método `AtualizarLivroAsync` do repositório.
5. **O Entity Framework faz o "De-Para":** O repositório busca o livro original que está guardado no banco de dados (o estado antigo). Ele pega o objeto que veio da tela e começa a sobrescrever o objeto do banco campo por campo:
 - título antigo era "A", agora vira "B".
 - gênero antigo era "X", agora vira "Y".
6. **A Persistência Real:** Ao chamar o `_context.SaveChangesAsync()`, o Entity Framework analisa o que mudou, monta uma query `SQL UPDATE Biblioteca.Livros SET Titulo = 'B', ... WHERE Id = 3` e executa no banco de dados.
7. **O Desfecho (Redirecionamento):** Com o banco atualizado com sucesso, a Controller responde ao navegador com um comando `RedirectToAction("Livro")`. O navegador limpa a tela de edição e joga o usuário de volta para a tabela de listagem, onde ele já vê os dados novos atualizados.



Passo a passo no projeto - Exemplo (Parte Prática)

1º Criar a nossa ViewModel de edição

Criar o arquivo `Models/EditarLivroViewModel.cs`

```
namespace Biblioteca.Models;

public class EditarLivroViewModel
{
    public int Id { get; set; }
    public string? Titulo { get; set; }
    public int NumPaginas { get; set; }
    public string? Genero { get; set; }
    public DateOnly DataPublicacao { get; set; }
    public int AutorId { get; set; }
}
```

Essa classe serve para transportar os dados da tela para a Controller com segurança, incluindo o ID do livro e o ID do autor selecionado.

2º Atualizar a Interface do Repositório

Alterar o arquivo `Repositories/LivroRepository.cs`

Aqui é necessário adicionar a assinatura do método de atualização para que a controller consiga enxergá-lo

```
public interface ILivroRepository
{
```

```

    Task<List<Livro>> BuscarTodosLivrosAsync();
    Task<bool> CriarLivroAsync(Livro livro, int AutorId);
    Task<bool> AtualizarLivroAsync(Livro Livro); // <- Adicionamos a assinatura
}

```

3º Implementar a Atualização no Repositório

Alterar o arquivo `Repositories/LivroRepository.cs`

Ajustamos (verificamos) o método de `BuscarTodosLivrosAsync` para carregar o objeto Autor do banco

```

// Atualizado com .Include para carregar o objeto Autor do banco
public async Task<List<Livro>> BuscarTodosLivrosAsync()
{
    return await _context.Livros.Include(x => x.Autor).ToListAsync();
}

```

Ainda no arquivo `LivroRepository` (caso não tenha um método com essa finalidade, crie um no arquivo) ajustamos o métodos que vai atualizar todos os campos no banco, do livro editado:

```

public async Task<bool> AtualizarLivroAsync(Livro Livro)
{
    // Usamos o Include para carregar também o Autor atual do banco se necessário
    // mas para atualizar a chave estrangeira precisamos tratar o Autor
    var livroBanco = await _context.Livros
        .Include(x => x.Autor)
        .FirstOrDefaultAsync(x => x.Id == Livro.Id);

    if (livroBanco == null) return false;

    // Atualiza TODOS os campos que vieram da tela
    livroBanco.Titulo = Livro.Titulo;
    livroBanco.Genero = Livro.Genero;
    livroBanco.NumPaginas = Livro.NumPaginas;
    livroBanco.DataPublicacao = Livro.DataPublicacao;

    // Atualiza o Autor
    livroBanco.Autor = Livro.Autor;

    await _context.SaveChangesAsync();
    return true;
}

```

4º Adicionar as Actions na Controller

Alterar o arquivo `Controllers/BibliotecaController.cs`

Precisamos adicionar os dois métodos `EditarLivro`

O GET irá carregar os dados atuais do livro para preencher a tela: (*Já implementamos na aula)

```
// GET: Biblioteca/EditarLivro/Id
[HttpGet]
public async Task<IActionResult> EditarLivro(int id)
{
    var livros = await _livroRepository.BuscarTodosLivrosAsync();
    var livro = livros.FirstOrDefault(x => x.Id == id);

    if (livro == null) return NotFound();

    // Transforma o livro do banco na ViewModel da tela
    var viewModel = new EditarLivroViewModel
    {
        Id = livro.Id,
        Titulo = livro.Titulo,
        NumPaginas = livro.NumPaginas,
        Genero = livro.Genero,
        DataPublicacao = livro.DataPublicacao,
        AutorId = livro.Autor?.Id ?? 0
    };

    // Alimenta o SelectList passando o AutorId atual para vir selecionado
    ViewBag.Autores = new SelectList(
        await _autorRepository.BuscarTodosAutoresAsync(),
        "Id",
        "Nome",
        viewModel.AutorId
    );

    return View(viewModel);
}
```

O POST recebe as alterações e redireciona de volta para a listagem:

```
// POST: Biblioteca/EditarLivro
[HttpPost]
public async Task<IActionResult> EditarLivro(EditarLivroViewModel livroViewModel)
{
    var autores = await _autorRepository.BuscarTodosAutoresAsync();
    var autorSelecioneado = autores.FirstOrDefault(a => a.Id == livroViewModel.AutorId);
```

```

// Converte a ViewModel de volta para o objeto de domínio Livro
Livro livro = new()
{
    Id = livroViewModel.Id,
    Titulo = livroViewModel.Titulo,
    Genero = livroViewModel.Genero,
    NumPaginas = livroViewModel.NumPaginas,
    DataPublicacao = livroViewModel.DataPublicacao,
    Autor = autorSelecionado
};

await _livroRepository.AtualizarLivroAsync(livro);

// Redireciona para a listagem principal de livros
return RedirectToAction("Livro");
}

```

5º Criar a View de Edição

Criar um arquivo `Views/Biblioteca/EditarLivro.cshtml`

Será um formulário HTML simples mapeado para as propriedades da ViewModel

```

@model Biblioteca.Models.EditarLivroViewModel

@{
    ViewData["Title"] = "Editar Livro";
}

<div class="container mt-4">
    <h1>Editar Livro</h1>
    <hr />

    <div class="row">
        <div class="col-md-6">
            <form asp-action="EditarLivro" method="post">
                <input type="hidden" asp-for="Id" />

                <div class="form-group mb-3">
                    <label asp-for="Titulo" class="control-label">Título d
o Livro</label>
                    <input asp-for="Titulo" class="form-control" />
                </div>

                <div class="form-group mb-3">
                    <label asp-for="Genero" class="control-label">Gênero</
label>

```

```

        <input asp-for="Genero" class="form-control" />
    </div>

    <div class="form-group mb-3">
        <label asp-for="NumPaginas" class="control-label">Número de Páginas</label>
        <input asp-for="NumPaginas" type="number" class="form-control" />
    </div>

    <div class="form-group mb-3">
        <label asp-for="DataPublicacao" class="control-label">Data de Publicação</label>
        <input asp-for="DataPublicacao" type="date" class="form-control" />
    </div>

    <div class="form-group mb-3">
        <label asp-for="AutorId" class="control-label">Autor</label>
        <select asp-for="AutorId" class="form-control" asp-items="ViewBag.Autores">
            <option value="">-- Selecione um Autor --</option>
        </select>
    </div>

    <div class="form-group mt-4">
        <button type="submit" class="btn btn-primary">Salvar Alterações</button>
        <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
    </div>
</form>
</div>
</div>
</div>

```

6º Ajustar o Botão de Edição na Listagem

Ajustar arquivo `Views/Biblioteca/Livro.cshtml` ou onde estiver a listagem de livros

O botão na tabela de listagem deve passar dinamicamente o `Id` do livro pela rota para disparar o fluxo.

```

<td>
    <a asp-action="EditarLivro" asp-route-id="@livro.Id" class="btn btn-sm

```

```
btn-primary">Editar</a>  
</td>
```

🤔 Qualquer dúvida converse com o professor em sala, ou então entre em contato pelo e-mail:

joao.noschang@colaborador.ifmt.edu.br